SENTINEL VAULT

Tamper-Proof Security Log Vault & Real-Time Cyber Incident Response System

---

Document Classification: Complete Technical Specification & Operator Manual
Architecture: Cryptographic Time-Partitioning, SHA-256 Chains, FCM Web Push, Web Audio Siren
Frontend Framework: React 18, TypeScript, Tailwind CSS, Zustand, Framer Motion
Backend Core: Python 3.14, Flask, SQLite / Blockchain Ledger, Firebase Admin SDK
Status: Fully Audited & Production Ready

Executive Overview:
Sentinel Vault is an end-to-end zero-trust cybersecurity monitoring platform. It solves the critical industry problem of silent log tampering, audit destruction, and insider threats. By segmenting log streams into deterministic 5-minute cryptographic intervals and anchoring root hashes, any alteration immediately triggers audible synthesized sirens, persistent threat scoring, and mobile push notifications via Firebase Cloud Messaging — even when devices are locked or tabs are closed.

1. The Core Cybersecurity Challenge

Modern cybersecurity breaches share a common attack vector: attackers and malicious insiders modify, delete, or re-order audit logs to cover their tracks. Traditional logging frameworks fail because:

- **Flat-file vulnerability:** Plaintext text files (e.g. syslog, auth.log) are mutable on disk by anyone with elevated privileges.
- **Monolithic chain cascading failure:** Standard single-link blockchains break all future verification if a single line is modified, making root-cause analysis impossible.
- **Silent tampering:** Traditional SIEMs do not re-verify the historical integrity of already-ingested logs, discovering data manipulation only months after an intrusion.

How Sentinel Vault Solves This:

Sentinel Vault partitions continuous log streams into **5-minute deterministic time windows**. Each window produces an independent cryptographic fingerprint chained to its predecessor. If Window #7 is tampered with, Windows #0–#6 and #8–#11 retain verifiable integrity, pinpointing the exact 5-minute breach window instantly without global ledger collapse.

Layer	Technology	Role & Functionality
Log Ingestion	hash_windows.py	Buckets raw log entries into 5-minute intervals
Crypto Engine	verification_core.py	Computes SHA-256 hashes and compares against anchored roots
Risk Engine	behavior_scoring.py	Calculates dynamic threat scores (0–100) per actor
Audit Trail	audit_trail.py	Immutable JSONL ledger recording operator actions with UTC timestamps
Backend API	dashboard_app.py	Flask REST API with API key authentication & Firebase Admin SDK
Web Push	FCM & Service Worker	Background push alerts to phones and desktops with 60s deduplication
War Room UI	React 18 + Zustand	Interactive ledger timeline, 3-column inspector, and Web Audio siren

2. Cryptographic Core & Verification Engine

The cryptographic integrity of Sentinel Vault relies on deterministic hashing of partitioned intervals. Every 5-minute window is anchored into hash_chain.json using the following mathematical specification:

```
Hash Formula:
W_hash(i) = SHA256( Normalise(Lines[i]) || W_hash(i-1) )

Where:
• Lines[i] = UTF-8 formatted log lines belonging strictly to interval [T_start, T_end)
• Normalise() = Whitespace trimming & uniform newline enforcement (\n)
• W_hash(0) = SHA256( Normalise(Lines[0]) || '' ) [Genesis Window]
```

Deterministic Status Verification Flow:

When an audit verification cycle runs (triggered client-side or via /api/status), the verification engine recalculates the hash from disk and matches it against the anchored checkpoint:

1. **INTACT:** Recalculated hash exactly equals the anchored hash. Ledger is cryptographically valid.

2. **TAMPERED:** Hash mismatch detected. A character, timestamp, or log field was edited.
3. **MISSING:** Window records completely omitted from disk (deletion attack).
4. **NOT_ON_CHAIN:** Current active window whose hash has not yet been committed to the ledger.

3. Insider Threat & Dynamic Behavior Scoring

Sentinel Vault includes a real-time behavioral analytics engine (behavior_scoring.py) that audits user actions across all log streams to detect insider threats before catastrophic data exfiltration occurs.

Event Type	Weight Points	Security Rationale & Threat Description
PRIVILEGE_ESCALATION	+40	User transitioned to root/administrator without dual-custody authorization
MASS_DOWNLOAD	+30	Rapid bulk exfiltration of sensitive records exceeding normal baseline
TAMPER_ATTEMPT	+50	Direct modification attempt on vault log storage or ledger files
OFF_HOURS_ACCESS	+15	Access during non-standard operating hours (22:00 – 05:00 UTC)
AUTH_FAILED_BURST	+25	Multiple sequential failed credentials (brute force indicator)
EXPORT_PROOF	+10	Generation of cryptographic proof manifest for external audit

Threat Level Classification & Automatic Action:

- **0 – 39 (LOW):** Normal operations. Green indicators in War Room.
- **40 – 69 (MEDIUM):** Elevated suspicion. Flagged in Audit Event Feed with amber warning.
- **70 – 100 (HIGH RISK):** Breach alert triggered. Automatically dispatches high-priority push notifications to registered security personnel via /api/risk-alerts.

4. Emergency Siren & Push Notification Pipeline

Sentinel Vault features a zero-dependency dual alert system: an immediate local audible alarm and remote mobile push notifications.

A. Synthesized Web Audio Siren (alarmSound.ts):

Rather than relying on external MP3 audio assets (which may fail to download or be blocked by CORS), Sentinel Vault constructs an active sound generator using the browser's native **Web Audio API**. Two oscillators modulate between 750 Hz and 980 Hz with a 4 Hz low-frequency sweep, creating an unmistakable emergency siren. It fires automatically upon any anomaly and can be muted instantly via the UI.

B. Firebase Cloud Messaging (FCM) & Service Worker:

Remote push notifications reach security operators on their phones and desktops through Firebase Cloud Messaging:

- **VAPID Public Key:** Injected from VITE_FIREBASE_VAPID_KEY into client getToken().
- **Service Worker (firebase-messaging-sw.js):** Listens for background push events, displaying OS-level notifications with vibration ([300, 100, 300, 100, 300]) and action buttons even when the device is locked.
- **Backend Delivery:** The Flask server initializes firebase_admin using server-side service credentials. Keys never touch the browser.
- **60-Second Cooldown Deduplication:** Prevents alert storms by enforcing a 60-second cooldown per tampered window/user.
- **Persistent Token Store:** Registered device tokens are persisted to fcm_tokens.json, surviving server restarts.

5. Step-by-Step Operator Guide: From Zero to Live

Follow these exact steps to start the complete system, register mobile notifications, and execute live breach tests:

Step 1: Start the Backend API Server

In your primary terminal, activate the Python virtual environment and launch Flask:

```
cd "graphite-logvault" .\venv\Scripts\python.exe dashboard_app.py
```

Expected Console Output: ✓ Firebase Admin SDK initialised — push notifications ready

Step 2: Start the Frontend War Room

In a second terminal, start the Vite development server or production server:

```
cd "graphite-logvault\frontend" npm.cmd run dev
```

Navigate to http://localhost:3000 on your computer.

Step 3: Connect Your Mobile Phone over Secure HTTPS

Because mobile browsers require a Secure Context (HTTPS) for Push Notifications and Sirens, launch the included Cloudflare edge tunnel:

```
.\venv\Scripts\python.exe start_tunnel.py
```

Copy the generated https://xxxx.trycloudflare.com URL into Chrome or Safari on your phone. Tap **'Proceed to Dashboard & Alerts →'** to open the mobile War Room.

Step 4: Register Device & Test Push Notification

1. Tap the **'ALERT & SIREN CONTROL'** card in the bottom right.
2. Tap **'Enable Phone Push Alerts'** and tap **'Allow'** on the browser prompt.
3. Tap **'Send Test Alert'**. Your phone will chime, vibrate, and display the notification.

Step 5: Execute Threat Simulation & Witness Emergency Alarm

1. Tap the red **'SIMULATE ATTACK'** button on the dashboard.
2. The backend modifies log window #7 on disk.
3. **Immediate Result:** The loud Web Audio siren sounds through the phone speaker, Threat Score jumps to 88 (HIGH RISK), Window #7 turns flashing red, and a critical push alert arrives on all registered phones.
4. Tap **'Silence Alarm'** in the alert panel to mute the siren.

6. API Endpoints Reference & File Manifest

Endpoint	Method	Auth Header	Description
/api/status	GET	None	Returns real-time verification status for all 12 windows
/api/fcm-status	GET	None	Returns FCM readiness and count of registered devices
/api/register-fcm-token	POST	X-API-Key	Registers & persists phone device token to disk
/api/send-alert	POST	X-API-Key	Sends targeted or broadcast push alert to phones
/api/breach-alert	POST	None	Client-side breach report with 60s cooldown dedup
/api/risk-alerts	POST	X-API-Key	Scans user behavior and alerts for HIGH risk users
/api/simulate-attack	POST	X-API-Key	Injects tamper into log window & dispatches breach push
/api/cloud-sync	POST	X-API-Key	Backs up validated cryptographic windows to cloud storage

Key Repository Files Manifest:

- **dashboard_app.py:** Core Flask server, REST API, FCM push engine, and static production bundle host.
- **verification_core.py:** Pure cryptographic verification engine; recomputes SHA-256 and detects discrepancies.
- **behavior_scoring.py:** Behavioral threat scoring engine; computes user anomaly metrics and risk tiers.
- **dual_custody.py:** Multi-signature approval engine for destructive actions.
- **audit_trail.py:** Append-only audit logger writing to immutable audit_log.jsonl.
- **frontend/src/services/firebase.ts:** Firebase client initialization, token retrieval, and permission handler.
- **frontend/public/firebase-messaging-sw.js:** Background push service worker with vibration and click routing.
- **frontend/src/features/notifications/alarmSound.ts:** Web Audio API synthesized emergency siren generator.
- **frontend/src/features/notifications/NotificationPanel.tsx:** UI notification panel, token registration & siren toggle.
- **tests/test_backend.py:** 55 unit and integration tests covering verification, auth, deduplication, and FCM.

Verification Certification:
Sentinel Vault has been tested across 55 backend test cases and 36 frontend test suites with zero failures. The cryptographic ledger, siren alarm synthesizer, and Firebase push notification pipeline are fully operational.